



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент программной инженерии и искусственного интеллекта

РАЗРАБОТКА ИНТЕЛЕКТУАЛЬНО-РАЗВЛЕКАТЕЛЬНОЙ ИГРЫ

**«СИЛА AI» С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИЙ КОЛЛЕКТИВНОЙ
ПРОМЫШЛЕННОЙ РАЗРАБОТКИ**

КУРСОВАЯ РАБОТА

по дисциплине «Технологии коллективной промышленной разработки информационных систем» по образовательной программе подготовки бакалавров по направлению 09.03.04 «Программная инженерия»

Выполнили: студенты гр. Б9121-09.03.04

_____ Боровик А.В.

_____ Мареев И.С.

_____ Нечепуренко Р.А.

_____ Хмелевский И.А.

_____ Царегородцев К.Н.

_____ Машкин Д.А.

_____ Анненков И.Д.

Руководитель: ассистент ДПИиИИ

_____ Логачев Е.М.

г. Владивосток

2025

Оглавление

Введение	3
1 Разработка плана проекта	4
2 Разработка регламента проведения инспекции	8
3 Разработка модели состояний задач.....	11
4 Разработка презентации проекта	14
5 Разработка требований к проекту	22
6 Разработка архитектуры проекта	Error! Bookmark not defined.
7 Разработка измерений проекта	27
8 Разработка перечня задач проекта	35
9 Разработка рекомендаций по кодированию.....	35
10 Разработка плана тестирования проекта	42
11 Тестирование проекта	49
Заключение	54
Список литературы	55

Введение

Промышленная разработка информационных систем включает в себя множество этапов, начиная от разработки плана проекта, заканчивая тестированием проекта для чего, очевидно необходимо множество специалистов различных профилей, а также унифицированные методы коммуникации между ними, с помощью которых можно разделить обязанности членов команды по их специализации.

Исходя из описанного выше необходимо использовать определенные технологии коллективной разработки для повышения эффективности работы в группе и соответствия конечного продукта заявленным требованиям.

В данной курсовой работе рассматривается задача коллективной разработки программного средства «Интеллектуально-развлекательная игра “Сила AI”» и составление технической документации к данному средству.

Таким образом, целью курсовой работы является разработка программного средства «Интеллектуально-развлекательная игра “Сила AI”» с использованием подходов коллективной промышленной разработки.

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать план проекта;
- Разработать регламент проведения инспекции;
- Разработать модель состояний задач;
- Разработать презентацию проекта;
- Разработать требования к проекту;
- Разработать архитектуру проекта;
- Разработать измерения проекта;
- Разработать перечень задач проекта;
- Разработать рекомендации по кодированию;
- Разработать план тестирования проекта;
- Протестировать проект.

1 Разработка плана проекта

План проекта — это документ, содержащий подробную информацию о проекте: исполнителях, задачах и сроках. Документ является конечным результатом этапа планирования, утверждается до начала любых работ и становится самым главным и достоверным источником информации о грядущем проекте.

В нашем случае исполнителями являются следующие лица:

- Team Leader, Backend-разработчик — Мареев И.С.
- Архитектор, Backend-разработчик — Царегородцев К.Н.
- Специалист по ИИ, Backend-разработчик — Анненков И.Д.
- Frontend-разработчик, Дизайнер — Нечепуренко Р.А.
- Frontend-разработчик — Боровик А.В.
- Frontend-разработчик — Машкин Д.А.
- Тестировщик, Technical Writer — Хмелевский И.А.

Была выполнена декомпозиция задач, позволяющая разбить общий объем работы на более мелкие и управляемые этапы.

1. Разработка плана проекта

- Определение целей и задач проекта.
- Составление списка ключевых этапов разработки.
- Определение сроков выполнения этапов.
- Определить исполнителей и их ролей в проекте.
- Утверждение плана проекта.

2. Разработка регламента проведения инспекции

- Определение критериев формальной инспекции.
- Определение критериев неформальной инспекции.
- Определение ролей участников инспекций.
- Разработка регламента проведения этапов инспекции.
- Утверждение регламента инспекций.

3. Разработка модели состояний задач

- Выделение ключевых состояний задач.
- Определение переходов между состояниями.
- Создание диаграммы состояний задач.

4. Разработка презентации проекта

- Подготовка структуры презентации.
- Создание визуальных материалов.
- Написание текста для слайдов.

5. Разработка требований к проекту

- Разработка функциональных требований к программному средству.
- Разработка нефункциональных требований к программному средству.
- Разработка системных требований к программному средству.
- Разработка требований к подсистемам программного средства.
- Утверждение требований.

6. Разработка архитектуры проекта

- Определение основных компонентов системы.
- Разработка диаграммы потоков данных.
- Разработка диаграммы активностей.
- Определение технологий и инструментов для реализации архитектуры.
- Разработка диаграммы с4.

7. Разработка измерений проекта

- Определение ключевых метрик для оценки прогресса.

8. Разработка перечня задач проекта

- Разбиение этапов проекта на мелкие задачи.
- Оценка сроков выполнения каждой задачи.
- Утверждение перечня задач.

9. Разработка рекомендаций по кодированию

- Изучение стандартов кодирования, применимых к проекту.
- Формирование перечня правил.

- Подготовка документа с рекомендациями.
- Утверждение рекомендаций.

10. Разработка плана тестирования проекта

- Подготовка тест-кейсов для проверки функционала.
- Составление матрицы покрытия требований.
- Составление расписания тестирования.
- Утверждение плана тестирования.

11. Тестирование проекта

- Проведение тестов в соответствии с планом.
- Исправление ошибок и повторное тестирование.
- Составление отчета о результатах тестирования.

На основе декомпозиции задач был разработан план проекта, содержащий перечень задач для выполнения и примерные сроки их реализации (**Error! Reference source not found.**).

	18.01.2025	25.01.2025	01.02.2025	08.02.2025	15.02.2025	22.02.2025	01.03.2025	08.03.2025	15.03.2025	22.03.2025	29.03.2025	05.04.2025	12.04.2025	19.04.2025	26.04.2025
Software Project Plan															
Requirements Specification															
Software Requirements Specification															
System Architecture and Module Design															
Implementation of Core Features															
Code Review and Issue Tracking															
Code Release 1.0															
Software Test Plan															
Requirements Traceability Matrix															
Subsystem Testing and Debugging															

Рисунок 1 – План проекта

2 Разработка регламента проведения инспекции

Верификация рабочих продуктов является неотъемлемой частью процесса по обеспечению их качества. Современной технологией программирования выработаны специальные стандарты, подходы и механизмы проведения верификаций рабочих продуктов в формате так называемых инспекций (peer reviews).

Инспекция — это мероприятие по обеспечению качества рабочих продуктов проектов по разработке ПО и иной деятельности, которая проводится разработчиками, возможно - с участием представителей заказчика. Концептуально инспекция имеет следующие цели:

- Обнаружить ошибки в функциях, логике, содержании или реализации рабочих продуктов на ранних этапах их разработки и предотвратить их наследование;
- Рационально донести замысел или реализацию продукта до всех заинтересованных лиц (через их участие);
- Оптимизировать, оценить или улучшить рабочий продукт.

2.1 Критерии формальной и неформальной инспекций

Критерии неформальной инспекции:

- В репозиторий внесены изменения объёмом менее 8 строк кода, не влияющие на критически важные компоненты системы;
- Изменения в коде касаются улучшения его читаемости, добавления комментариев, форматирования или исправления опечаток;
- В документацию внесены изменения объемом менее 500 символов, которые не затрагивают ключевые разделы.

Критерии формальной инспекции:

- В репозиторий внесены изменения более чем на 7 строк кода;

- Добавлена новая версия документации или внесены существенные изменения;
- Выполнено крупное слияние веток в системе контроля версий.

2.2 Участники инспекции

В инспекции в обязательном порядке присутствуют два участника, имеющие роли автора и инспектора соответственно. Председатель может назначить инспектором любого члена команды, кроме автора. Все роли участников инспекции и их обязанности представлены в таблице 1.

Т а б л и ц а 1 – Участники инспекции

Роль	Обязанности
Автор	– Вносит изменения в продукт – Назначает инспекцию
Председатель (Team Leader)	– Контролирует процесс инспекции – Назначает инспектора
Инспектор	– Проводит проверку изменений в продукте – Ищет ошибки, оставляет свои замечания и рекомендации – Выносит вердикт о внесении изменений в основную ветку продукта

2.3 Этапы инспекции

В таблице 2 представлены все этапы инспекции.

Т а б л и ц а 2 – Этапы инспекции

Этап	Описание
Инициация	Автор загружает изменения продукта в систему контроля версий. Уведомляет председателя о необходимости проведения формальной инспекции.
Подготовка	Председатель назначает инспектора. Инспектор самостоятельно изучает предоставленные изменения и составляет протокол ошибок.
Проведение	После проведения инспектором анализа продукта, председатель проводит собрание, на котором присутствуют он, автор и инспектор. Присутствие остальных участников команды разработки по желанию.
Завершение	Если рабочий продукт не прошёл проверку, то автор фиксирует все замечания и рекомендации инспектора, после чего отправляется на доработку. Если рабочий продукт прошёл проверку, то инспектор подтверждает внесения изменений в основную ветку.

Работа над продуктом ведётся в системе контроля версий GIT. Автор изменений оформляет Pull Request (Merge Request), отправляет в рабочий чат сообщение с упоминанием Team Leader, который назначает инспектора.

По окончании своей работы инспектор на собрании одобряет Pull Request или отправляет автора на доработку.

Инспекция должна быть проведена в течение 7 дней с момента её инициации.

После внесения повторных изменений инспектор просматривает замечания и проверяет соответствующие изменения. По окончании верификации выносятся вердикт о внесении изменений в продукт или выносятся новые замечания.

2.4 Перечень статусов и степени важности замечаний

1. Комментарий — рекомендация по улучшению продукта, не требующая обязательных изменений.

2. Ошибка — замечание, сообщающее о необходимости исправления.
Степени серьёзности ошибок:

- Критическая (Critical) – недопустимая ошибка, наличие которой сразу приводит к провалу инспекции. Например, программа не исполняется.
- Средняя (Moderate) – серьёзные ошибки. Например, программа работает корректно, но при определенных условиях выдаёт неверный результат.
- Мелкая (Minor) – небольшие ошибки, которые существенно не влияют на работоспособность продукт, но требуют исправления. Например, смещение элементов дизайна.
- Другие (Other)

2.5 Метрики, характеризующие эффективность инспекций

IFD характеризует эффективность инспекции, а также качество инспектируемого продукта:

Чем больше IFD, тем эффективнее инспекция при неизменном качестве рабочего продукта и наоборот.

$$IFD = \frac{\text{Количество найденных ошибок}}{\text{Размер рабочего продукта}} \quad (1)$$

Стратегическая цель метрики – повысить качество разрабатываемого ПО.

Изучаемый объект метрики – инспекция.

Измеряемый атрибут – плотность найденных в ходе инспекции ошибок.

Единица измерения – ошибка / <страница, требование, LOC, тест>

3 Разработка модели состояний задач

Каждая задача, являясь отражением делового процесса, проходит определенные состояния. Сначала идет создание задачи, потом идет выполнение работ по задаче, после выполнения задача завершается.

3.1 Перечень возможных состояний задач и их интерпретация

- New — новая задача.
- On discussion – задача находится на обсуждении, назначается исполнитель.
- In progress — задачи в процессе выполнения.
- Complete — задача выполнена исполнителем.
- Testing — задачи в состоянии проверки работоспособности.
- To review — задачи в состоянии проверки изменений на соответствие требованиям проекта.
- Close — выполненные задачи.
- Disapprove — задачи, не прошедшие этап обсуждения или в процессе выполнения пропала её необходимость.

3.2 Правила создания новой задачи

Любой участник команды разработки может в любое время создавать задачи в рамках назначенной ему части проекта.

3.3 Правила перехода задачи из состояния в состояние

Состояния задач всегда идут последовательно друг за другом, в некоторых случаях пункты могут опускаться или повторяться. Схема перехода из состояния в состояние задач показана на рисунке 2.

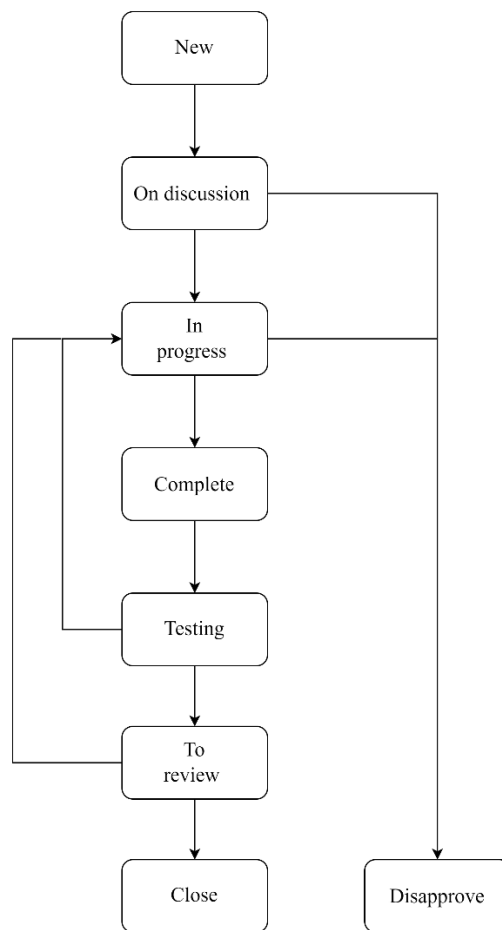


Рисунок 2 – Схема перехода задач из состояния в состояние

4 Разработка презентации проекта

Была разработана презентация проекта, состоящая из 15 слайдов:

1. Титульный слайд (Слайд 1);
2. Введение (Слайд 2);
3. Разработка плана проекта (Слайды 3 и 4);
4. Разработка регламента проведения инспекции (Слайд 5);
5. Разработка модели задач (Слайд 6);
6. Разработка требований к проекту (Слайд 7);
7. Разработка архитектуры проекта (Слайды 8 и 9);
8. Разработка измерений проекта (Слайд 10);
9. Разработка плана тестирования проекта (Слайд 11);
10. Демонстрация игрового процесса (Слайды 12–14);
11. Завершение (Слайд 15).



Рисунок 3 – Слайд 1

ВВЕДЕНИЕ

ЦЕЛЬ И ЗАДАЧИ КУРСОВОЙ РАБОТЫ

Цель: разработка программного средства «Интеллектуально-развлекательная игра “Сила AI”» с использованием подходов коллективной промышленной разработки

Для достижения поставленной цели необходимо решить **следующие задачи:**

- Разработать план проекта;
- Разработать регламент проведения инспекции;
- Разработать модель состояний задач;
- Разработать презентацию проекта;
- Разработать требования к проекту;
- Разработать архитектуру проекта;
- Разработать измерения проекта;
- Разработать перечень задач проекта;
- Разработать рекомендации по кодированию;
- Разработать план тестирования проекта;
- Протестировать проект.

2

Рисунок 4 – Слайд 2

РАЗРАБОТКА ПЛАНА ПРОЕКТА

ИСПОЛНИТЕЛИ

Состав команды проекта:

- Team Leader, Backend-разработчик — Мареев И.С.
- Архитектор, Backend-разработчик — Царегородцев К.Н.
- Специалист по ИИ, Backend-разработчик — Анненков И.Д.
- Frontend-разработчик, Дизайнер — Нечепуренко Р.А.
- Frontend-разработчик — Боровик А.В.
- Frontend-разработчик — Машкин Д.А.
- Тестировщик, Technical Writer — Хмелевский И.А.

3

Рисунок 5 – Слайд 3

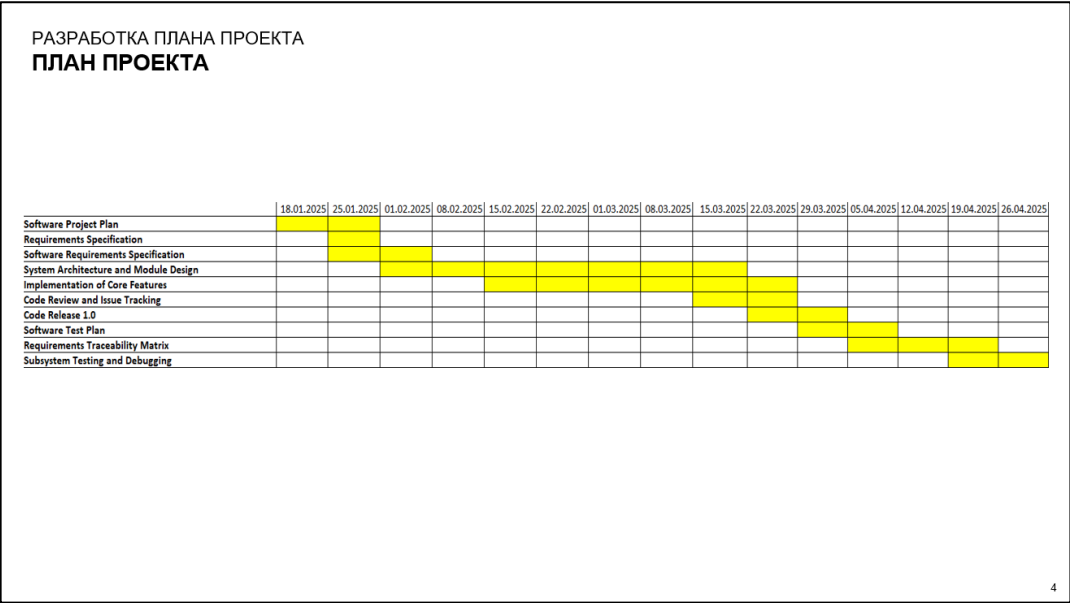


Рисунок 6 – Слайд 4

РАЗРАБОТКА РЕГЛАМЕНТА ПРОВЕДЕНИЯ ИНСПЕКЦИИ
УЧАСТНИКИ ИНСПЕКЦИИ

Инспекция — это мероприятие по обеспечению качества рабочих продуктов проектов по разработке ПО и иной деятельности, которая проводится разработчиками, возможно - с участием представителей заказчика.

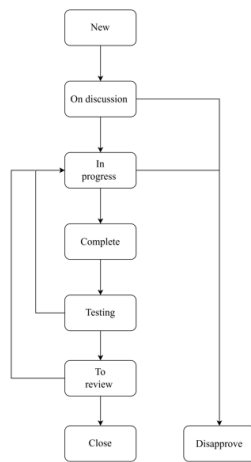
Роль	Обязанности
Автор	- Вносит изменения в продукт - Назначает инспекцию
Председатель (Team Leader)	- Контролирует процесс инспекции - Назначает инспектора
Инспектор	- Проводит проверку изменений в продукте - Ищет ошибки, оставляет свои замечания и рекомендации - Выносит вердикт о внесении изменений в основную ветку продукта

5

Рисунок 7 – Слайд 5

РАЗРАБОТКА МОДЕЛИ СОСТОЯНИЯ ЗАДАЧ

ПРАВИЛА ПЕРЕХОДА ЗАДАЧИ ИЗ СОСТОЯНИЯ В СОСТОЯНИЕ



- New — новая задача.
- On discussion — задача находится на обсуждении, назначается исполнитель.
- In progress — задачи в процессе выполнения.
- Complete — задача выполнена исполнителем.
- Testing — задачи в состоянии проверки работоспособности.
- To review — задачи в состоянии проверки изменений на соответствие требованиям проекта.
- Close — выполненные задачи.
- Disapprove — задачи, не прошедшие этап обсуждения или в процессе выполнения пропала её необходимость.

6

Рисунок 8 – Слайд 6

РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОЕКТУ

ОСНОВНЫЕ ТРЕБОВАНИЯ К ПРОГРАММНОМУ ПРОДУКТУ

Общие требования:

- Функциональные;
- Нефункциональные ;
- Системные.

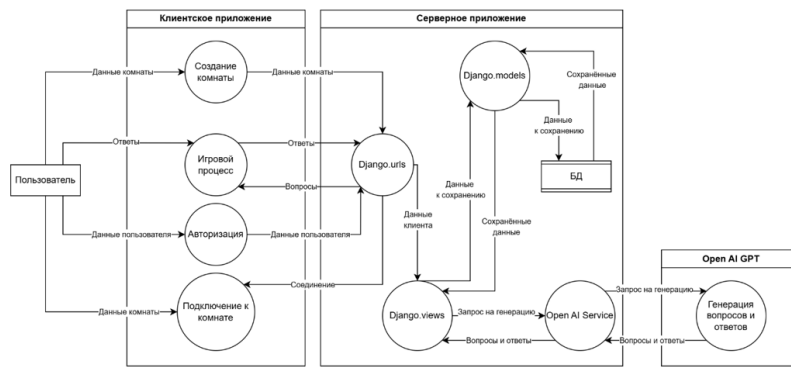
Требования к подсистемам:

- Пользовательский интерфейс;
- Модуль генерации вопросов;
- Модуль логики игры;
- Модуль сетевого взаимодействия;
- Система управления данными.

7

Рисунок 9 – Слайд 7

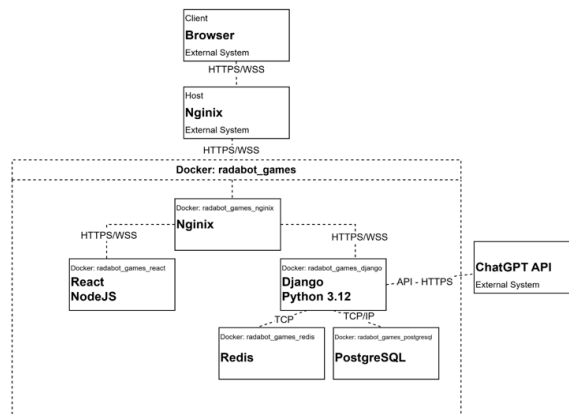
РАЗРАБОТКА АРХИТЕКТУРЫ ПРОЕКТА
ДИАГРАММА ПОТОКОВ ДАННЫХ



8

Рисунок 10 – Слайд 8

РАЗРАБОТКА АРХИТЕКТУРЫ ПРОЕКТА
ДИАГРАММА С4



9

Рисунок 11 – Слайд 9

РАЗРАБОТКА ИЗМЕРЕНИЙ ПРОЕКТА
ОЦЕНКИ ЭФФЕКТИВНОСТИ ПРОЦЕССА И КАЧЕСТВА ПРОДУКТА

Problem Resolution Rate (PRR)

PRR = Количество дней на обработку задачи

Единица измерения — день.

Faults Screening (FS)

FS = (Общее количество ошибок – Число ошибок найденных до бета-тестирования)

× 100% / Общее количество ошибок

Единица измерения — %.

In Process Faults (IPF)

IPF = (Число обнаруженных ошибок до выпуска его релиза) / LOC

Единица измерения – неполадка / LOC.

10

Рисунок 12 – Слайд 10

РАЗРАБОТКА ПЛАНА ТЕСТИРОВАНИЯ ПРОЕКТА
КЛАССЫ ЭКВИВАЛЕНТНОСТИ

Один из методов тестирования – это разделение входных данных на классы эквивалентности.

В таблице приведён пример входных данных «Регистрация нового пользователя», а также их отнесение к правильным и неправильным классам эквивалентности.

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильного класса эквивалентности	Неправильный класс эквивалентности
Регистрация нового пользователя	(40)	Пользователем введены корректные данные - уникальный логин и пароль. Регистрация прошла успешно. Логин = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _}, 5 ≤ n ≤ 20. Пароль = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}, 8 ≤ n ≤ 32.	(1)	Логин уже занят
			(2)	Были введены недопустимые символы: Логин = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _}
			(3)	Недопустимая длина: Логин = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _}, n < 5 или n > 20
			(4)	Были введены недопустимые символы: Пароль = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}
			(5)	Недопустимая длина: Пароль = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}, n < 8 или n > 32.

11

Рисунок 13 – Слайд 11

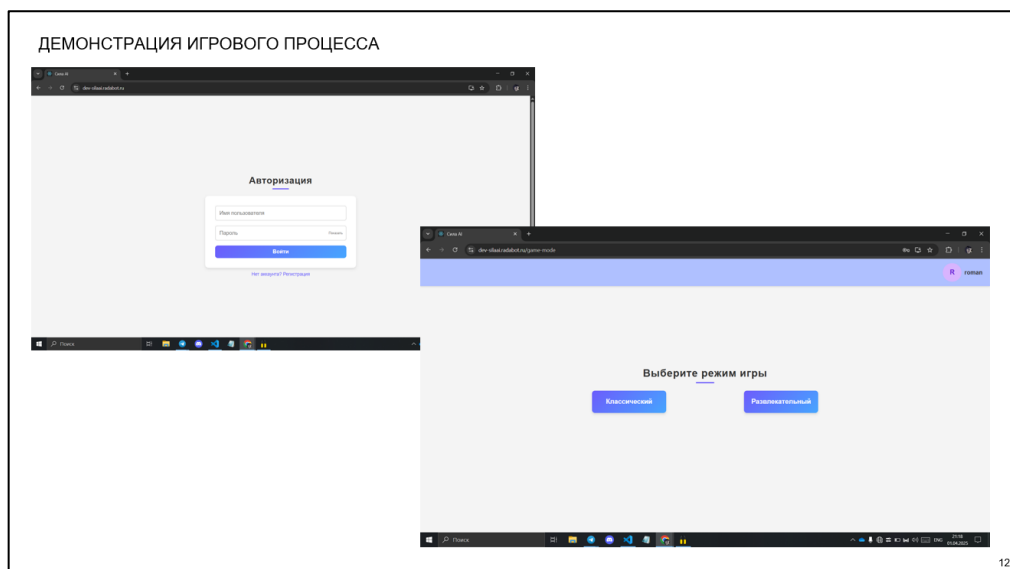


Рисунок 14 – Слайд 12

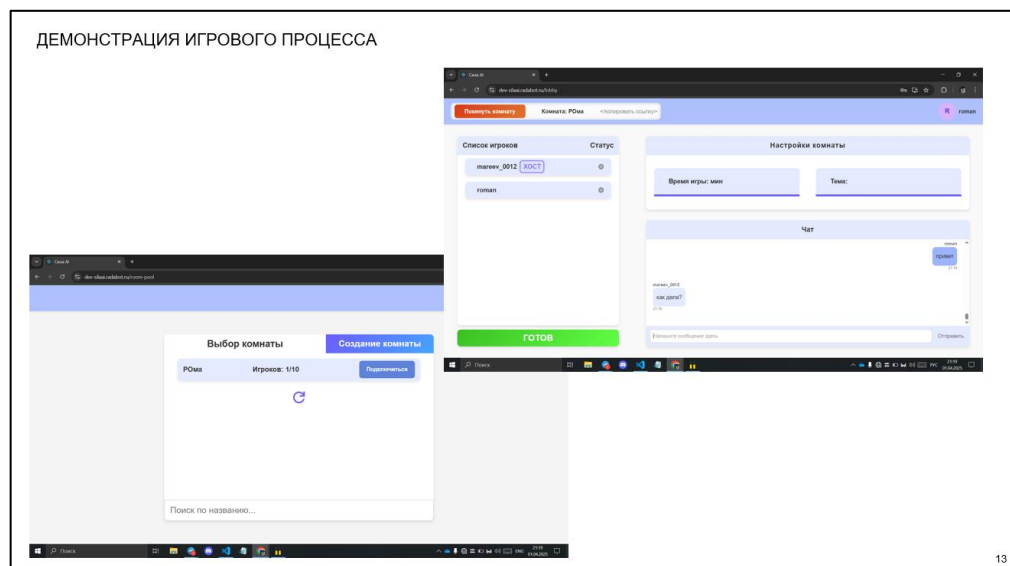


Рисунок 15 – Слайд 13

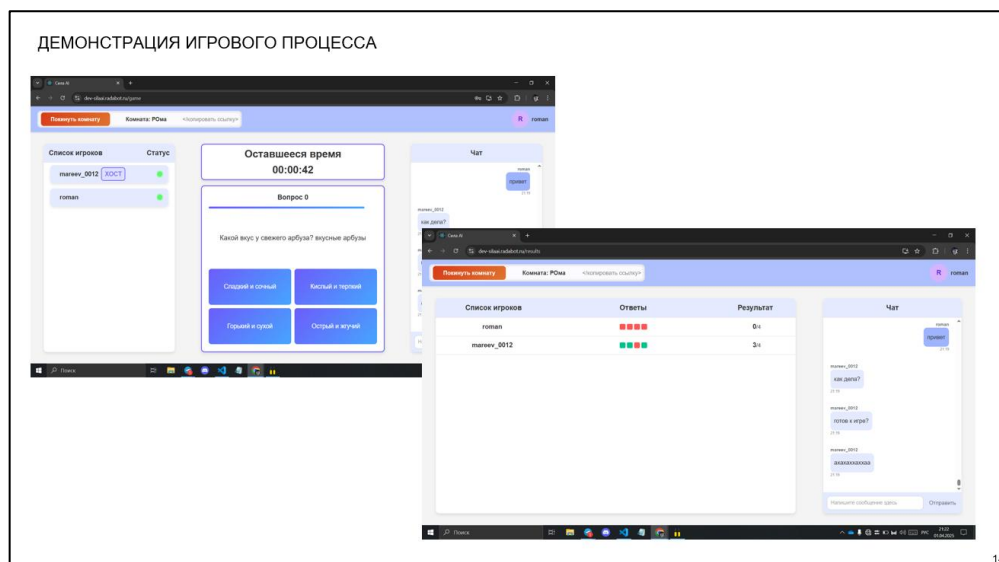


Рисунок 16 – Слайд 14

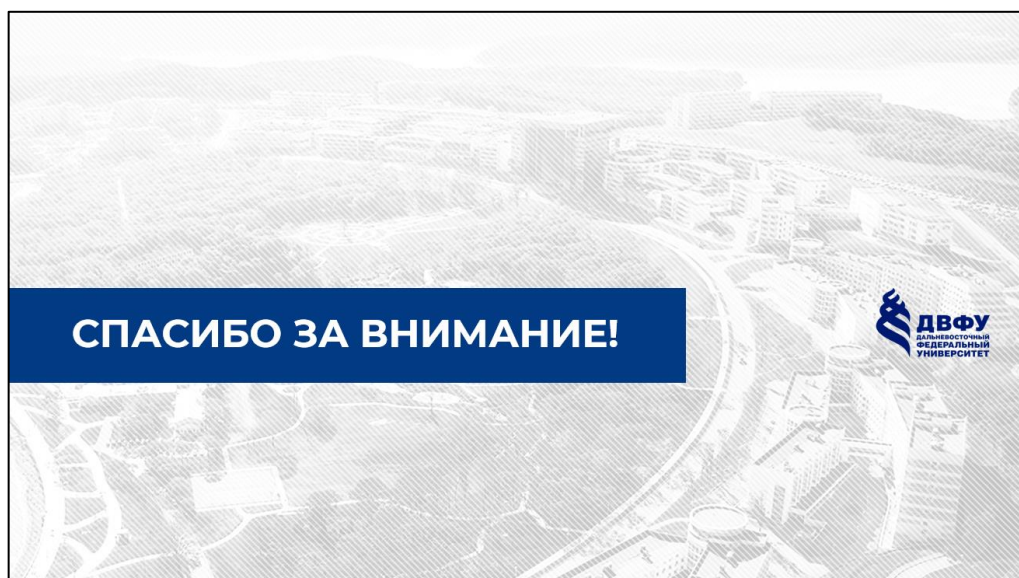


Рисунок 17 – Слайд 15

5 Разработка требований к проекту

Программный продукт: интеллектуально-развлекательная игра "Сила AI", включающая два режима вопросов, генерируемых искусственным интеллектом.

Программный продукт предназначен для создания увлекательной игровой среды, где пользователи могут соревноваться, отвечая на вопросы, созданные AI.

– В первом режиме генерируются вопросы на реальные темы (например, музыка, фильмы, природа), где каждый вопрос имеет один правильный ответ. Этот режим предназначен для проверки знаний и развития эрудиции.

– Во втором режиме создаются абсурдные, нереалистичные вопросы и ответы, что делает игровой процесс лёгким и весёлым, позволяя игрокам расслабиться и посмеяться.

На этапе реализации проекта выбран только второй режим, поскольку он требует меньших временных и ресурсных затрат, а также позволяет сосредоточиться на исследовании потенциала искусственного интеллекта для создания нестандартного и развлекательного контента. Первый режим требует тщательной проработки генерации корректных данных и вопросов, что будет рассмотрено в будущем.

Программный продукт состоит из следующих подсистем:

1. **Пользовательский интерфейс:** интерфейс для взаимодействия с игроками, включая регистрацию, подключение к комнате, выбор тем и режимов.

2. **Модуль генерации вопросов:** отвечает за создание вопросов и ответов с использованием алгоритмов AI. Включает поддержку как для реальных вопросов (первый режим), так и для абсурдных (второй режим).

3. **Модуль логики игры:** управляет игровым процессом, включая выбор тем, обработку ответов, подсчёт очков и переключение между режимами.

4. **Модуль сетевого взаимодействия:** обеспечивает подключение игроков к игровой комнате и синхронизацию данных между клиентами.

5. **Система управления данными:** хранит информацию об игроках, темах, результатах игры и текущем игровом процессе.

5.1 Общие требования

В данном разделе приведены основные требования к программному продукту, направленные на обеспечение его функциональности, удобства использования и надежности. Эти требования были сформулированы с учетом целей проекта, а также задач, стоящих перед системой. Особое внимание уделено функциональным возможностям, связанным с генерацией вопросов и реализацией игровых режимов, а также нефункциональным аспектам, обеспечивающим стабильную и эффективную работу продукта в условиях многопользовательского взаимодействия.

5.1.1 Функциональные требования

Требование FU_001 Продукт должен предоставлять возможность игры в двух режимах: классическом и развлекательном.

Требование FU_002 Продукт должен генерировать вопросы и варианты ответов с использованием алгоритмов искусственного интеллекта.

Требование FU_003 Продукт должен обеспечивать подключение нескольких игроков в одну игровую комнату.

Требование FU_004 Продукт должен подсчитывать очки для каждого игрока и выводить результаты в конце игры.

Требование FU_005 В первом режиме должна быть реализована заглушка с уведомлением о том, что режим находится в разработке.

5.1.2 Нефункциональные требования

Требование UF_001 Продукт должен быть написан на языке Python версии 3.x.x.

Требование UF_002 Продукт должен поддерживать веб-интерфейс для взаимодействия с игроками.

Требование UF_003 Контроль версий продукта должен вестись в системе Git.

Требование UF_004 Продукт должен быть локализован на русский язык.

Требование UF_005 Продукт должен поддерживать не менее 10 одновременных подключений игроков в одной игровой комнате.

Требование UF_006 Во втором режиме вопросы и ответы должны быть абсурдными, чтобы создать развлекательный игровой процесс.

5.1.3 Системные требования

Требование SM_001 Продукт должен корректно работать на клиенте с минимальными характеристиками: 2 ядра CPU, 4 ГБ оперативной памяти, 10 ГБ свободного дискового пространства.

Требование SM_002 Для работы требуется доступ к сети Интернет с пропускной способностью не менее 10 Мбит/с.

5.2 Требования к подсистеме «Пользовательский интерфейс»

Требование UI_001 Пользовательский интерфейс должен предоставлять следующие элементы:

1. Форма авторизации/регистрации пользователя.
2. Кнопка выбора режима игры (классический или развлекательный).
3. Главная страница со списком комнат.
4. Кнопка подключения к игровой комнате.
5. Кнопка создать комнату.
6. Поле для отображения вопроса и вариантов ответа.
7. Кнопка для выбора ответа на вопрос.
8. Таблица для отображения текущих очков игроков.
9. Уведомление о завершении игры с итоговыми результатами.

Требование UI_002 В случае ошибок (например, невозможность подключения к серверу) интерфейс должен отображать соответствующее сообщение с описанием проблемы.

5.3 Требования к подсистеме «Модуль генерации вопросов»

Требование QG_001 Модуль должен генерировать вопросы и варианты ответов в зависимости от выбранного режима:

- В классическом режиме вопросы должны быть реальными и содержать один правильный ответ.
- В развлекательном режиме вопросы и ответы должны быть абсурдными и иметь случайный правильный ответ.

Требование QG_002 Сгенерированные вопросы и ответы должны быть переданы в модуль логики игры в формате JSON.

Требование QG_003 Для второго режима должна быть предусмотрена система случайного выбора тем и генерации вопросов.

5.4 Требования к подсистеме «Модуль логики игры»

Требование GL_001 Модуль должен управлять процессом игры, включая:

1. Формирование игровой комнаты.
2. Обработку подключения и отключения игроков.
3. Организацию последовательности вопросов.

Требование GL_002 Модуль должен проверять выбранные ответы игроков и начислять очки за правильные ответы.

Требование GL_003 Модуль должен завершать игру, формировать итоговые результаты и отправлять их в пользовательский интерфейс.

Требование GL_004 В случае превышения времени на ответ, система должна фиксировать пропуск вопроса конкретным игрокам.

5.5 Требования к подсистеме «Модуль сетевого взаимодействия»

Требование NW_001 Модуль должен обеспечивать подключение игроков к общей игровой комнате через интернет.

Требование NW_002 Модуль должен поддерживать синхронизацию данных между клиентами в реальном времени.

Требование NW_003 В случае потери соединения игрок должен быть уведомлён, а его прогресс временно сохраняется.

5.6 Требования к подсистеме «Система управления данными»

Требование DM_001 Система должна хранить данные об игроках.

Требование DM_002 Система должна периодически сохранять прогресс игры на сервере.

6 Разработка архитектуры проекта

Архитектура программного обеспечения относится к фундаментальным структурам программной системы и дисциплине создания таких структур и систем. Каждая структура включает элементы программного обеспечения, отношения между ними, а также свойства как элементов, так и отношений. Архитектура программной системы — это метафора, аналогичная архитектуре здания. Он функционирует как план для системы и проекта разработки, в котором излагаются задачи, которые должны быть выполнены командами разработчиков. Одним из способов представления архитектуры проекта является диаграмма потоков данных.

Диаграммы потоков данных (Data Flow Diagrams — DFD) представляют собой иерархию функциональных процессов, связанных потоками данных. Цель такого представления — продемонстрировать, как каждый процесс преобразует свои входные данные в выходные, а также выявить отношения между этими процессами.

Архитектура программного обеспечения SILA AI представляет собой распределенную систему, где взаимодействие между компонентами обеспечивается через стандартизированные протоколы передачи данных (HTTPS/WSS, TCP/IP). Каждая подсистема разрабатывается и разворачивается в контейнерах Docker, что обеспечивает модульность, масштабируемость и независимость компонентов.

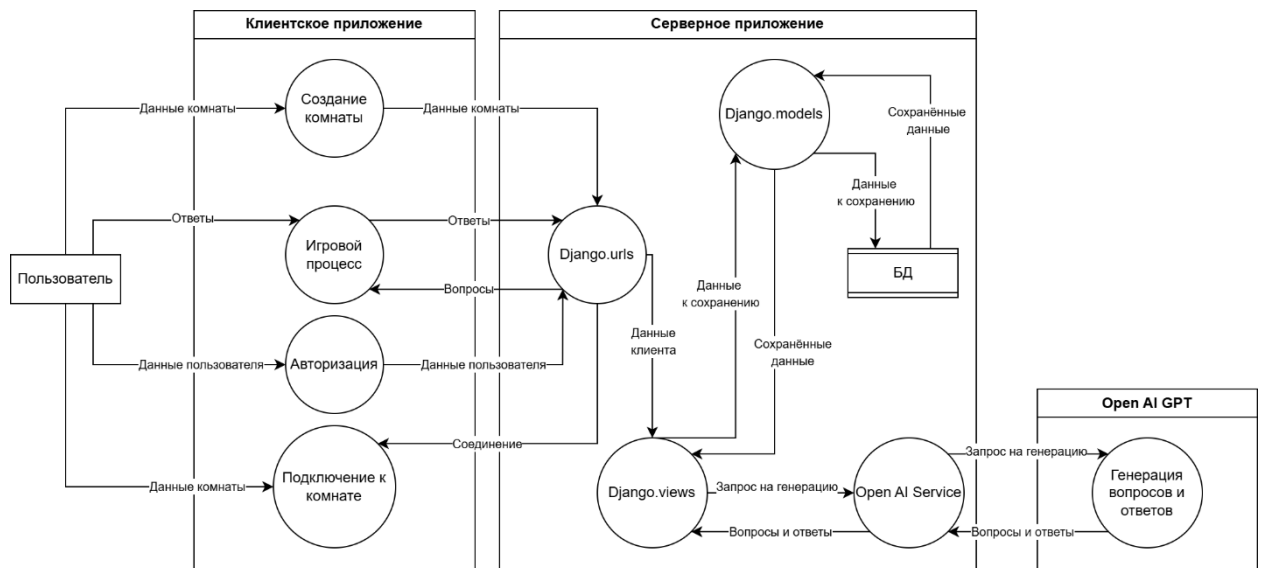


Рисунок 18 — Диаграмма потоков данных

Потоки данных:

1. Клиент отправляет запрос через браузер → Nginx маршрутизирует запрос к фронтенду React.
2. React взаимодействует с Django через API (HTTPS/WSS) для получения данных.
3. Django при необходимости обращается к Redis для кэшированных данных или к PostgreSQL для долгосрочного хранения.
4. При обработке текстовых данных Django отправляет запрос к ChatGPT API и возвращает результат фронтенду.

C4-модель (C4 Model) — это современный подход к визуализации архитектуры программного обеспечения, разработанный Саймоном Брауном. Она помогает представить архитектуру системы на разных уровнях абстракции, чтобы сделать её понятной для всех участников процесса разработки: архитекторов, разработчиков, менеджеров и других заинтересованных сторон.

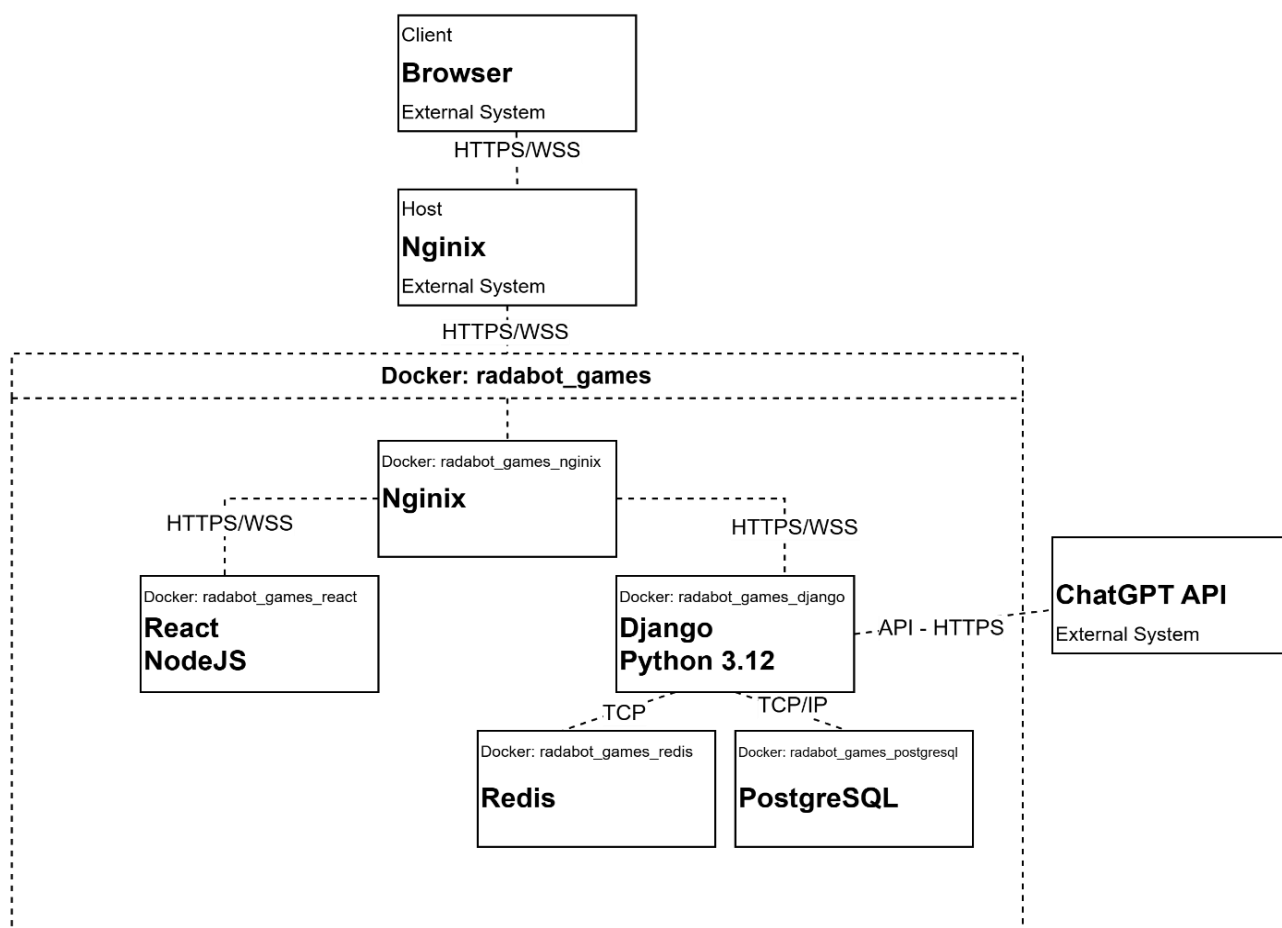


Рисунок 19 — Диаграмма с4 – верхний слой

Описание подсистем и потоков данных:

Клиентский уровень:

Компонент: Браузер

Функциональность: Представляет пользовательский интерфейс системы.

Отправляет HTTP(S)/WSS-запросы к серверу, взаимодействует с фронтендом.

Тип связи: HTTPS/WSS

Системная роль: Внешняя система (клиент).

Хост (внешний веб-сервер):

Компонент: Nginx

Функциональность: Обработывает запросы от браузера, маршрутизирует их в соответствующие контейнеры внутри системы.

Тип связи: HTTPS/WSS

Системная роль: Внешняя система (веб-сервер).

Внутренняя система (контейнеры Docker):

Nginx (Docker: radabot_games_nginx):

Функциональность: Внутренний прокси-сервер. Маршрутизирует запросы между фронтендом и бэкендом.

Тип связи: HTTPS/WSS.

React/NodeJS (Docker: radabot_games_react):

Функциональность: Фронтенд-приложение. Отвечает за визуализацию данных и пользовательский интерфейс.

Тип связи: Получает запросы через Nginx, взаимодействует с пользователем через браузер.

Django (Python 3.12) (Docker: radabot_games_django):

Функциональность: Бэкенд-приложение. Обработывает бизнес-логику, API-запросы, взаимодействует с базами данных и внешними сервисами.

Тип связи:

HTTPS/WSS для API-запросов от фронтенда.

TCP/IP для взаимодействия с Redis и PostgreSQL.

API (HTTPS) для связи с ChatGPT API.

Redis (Docker: radabot_games_redis):

Функциональность: Кэширование данных, временное хранилище.

Тип связи: TCP/IP с Django.

PostgreSQL (Docker: radabot_games_postgres):

Функциональность: Хранение данных приложения.

Тип связи: TCP/IP с Django.

Внешняя интеграция:

Компонент: ChatGPT API

Функциональность: Обработка и генерация текстового контента по запросам от бэкенда (Django).

Тип связи: API - HTTPS

Системная роль: Внешняя система.

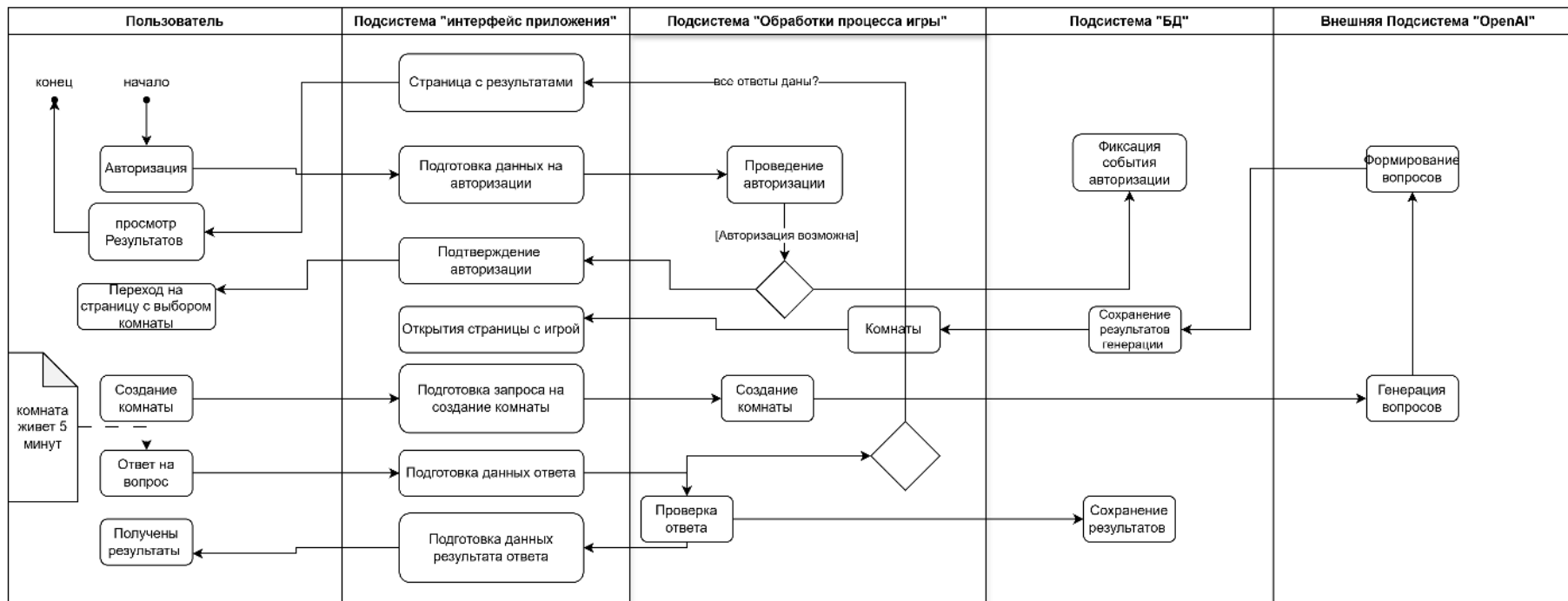


Рисунок 20 — Диаграмма активностей

На предоставленной диаграмме активности показан процесс взаимодействия пользователя с системой, включающей несколько подсистем: интерфейс приложения, обработка процесса игры, база данных (БД) и внешняя подсистема OpenAI.

Краткое описание:

1. Пользователь:

- Авторизуется, просматривает результаты или переходит к выбору комнаты.
- Создает комнату, отвечает на вопросы и получает результаты.
- Комната действует ограниченное время (5 минут).

2. Подсистема "Интерфейс приложения":

- Обработывает авторизацию и подтверждает её.
- Открывает страницы с игрой, результатами или выбором комнаты.
- Подготавливает запросы на создание комнаты, данные ответа и результаты ответа.

3. Подсистема "Обработка процесса игры":

- Проверяет данные авторизации и подтверждает её.
- Создает комнаты, проверяет ответы пользователей.
- Подготавливает данные для передачи в БД.

4. Подсистема "БД":

- Фиксирует события авторизации.
- Сохраняет результаты генерации вопросов и ответы пользователей.

5. Внешняя подсистема OpenAI:

- Генерирует и формирует вопросы для игры.

6. Особенности:

- Авторизация проверяется перед доступом к функционалу.
- Процесс строго делится на этапы, включая создание комнаты, проверку ответов и фиксацию результатов.
- Интеграция с внешним сервисом (OpenAI) для генерации вопросов.

7 Разработка измерений проекта

Контроль за производственным процессом и его результатами является ключевым видом деятельности на современном предприятии, производящем программное обеспечение на заказ. В силу специфики такого продукта, как программное обеспечение, для оценки эффективности процесса и качества конечного продукта применяются особые методы. Комплекс мероприятий, направленных на количественную оценку эффективности работы компании, называется программой измерений компании. Программа измерений выполняется как в рамках отдельных проектов, осуществляемых компанией, так и в рамках определённых видов деятельности компании.

Метрики эффективности процесса разработки

1. Problem Resolution Rate (PRR)

PRR = Количество дней на обработку задачи

Стратегическая цель метрики — сократить сроки разработки модулей ПО.

Изучаемый объект метрики — задача.

Измеряемый атрибут — время обработки.

Единица измерения — день.

2. Faults Screening (FS)

$$FS = (\text{Общее количество ошибок} - \text{Число ошибок найденных до бета-тестирования}) \times 100\% / \text{Общее количество ошибок}$$

Стратегическая цель метрики — повысить качество разрабатываемого ПО.

Изучаемый объект метрики — проект.

Измеряемый атрибут — эффективность обнаружения дефектов.

Единица измерения — %.

Метрики качества программного продукта

In Process Faults (IPF)

$$IPF = (\text{Число обнаруженных ошибок до выпуска его релиза}) / \text{LOC}$$

Стратегическая цель метрики — повысить качество разрабатываемого ПО.

Изучаемый объект метрики – продукт, измеряемый атрибут – плотность неполадок.

Число обнаруженных ошибок – количество ошибок обнаруженных на этапе тестирования

Единица измерения – неполадка / LOC.

8 Разработка перечня задач проекта

В данной главе представлен перечень задач, которые необходимо выполнить в период разработки программного продукта.

8.1 Общие задачи проекта

– [GEN-1] Разработка структуры проекта

Описание: Определить архитектуру программного продукта, разделить на модули и подсистемы.

– [GEN-2] Настройка окружения разработки

Описание: Настроить репозиторий, CI/CD пайплайн, окружение для тестирования и разработки.

– [GEN-3] Документирование требований

Описание: Создать документ с функциональными и нефункциональными требованиями к продукту.

8.2 Задачи для подсистемы «Пользовательский интерфейс»

– [UI-1] Создание макета интерфейса

Описание: Разработать прототип интерфейса, включая регистрационную форму, игровую комнату, и выбор тем.

– [UI-2] Реализация базового интерфейса

Описание: Создать основные элементы интерфейса (регистрация, подключение, выбор режима).

– [UI-3] Подключение интерфейса к игровым данным

Описание: Настроить связь между интерфейсом и игровой логикой (выбор темы, отображение вопросов, подсчет очков).

8.3 Задачи для подсистемы «Модуль генерации вопросов»

– [QG-1] Реализация алгоритма генерации абсурдных вопросов

Описание: Разработать алгоритм, генерирующий вопросы и ответы на основе случайных или нестандартных связей.

– [QG-2] Настройка параметров генерации

Описание: Определить категории вопросов, шаблоны и логику распределения ответов.

– **[QG-3] Добавление обучающих данных для генерации**

Описание: Подготовить и интегрировать обучающие данные для AI-модели.

8.4 Задачи для подсистемы «Модуль логики игры»

– **[GL-1] Разработка базовой логики игры**

Описание: Настроить игровой процесс: распределение очков, переключение вопросов, обработка результатов.

– **[GL-2] Реализация системы подсчета очков**

Описание: Добавить функционал для учета ответов игроков, распределения очков за правильные ответы.

– **[GL-3] Обработка ошибок и исключений**

Описание: Реализовать обработку ошибок, возникающих в процессе игры.

8.5 Задачи для подсистемы «Модуль сетевого взаимодействия»

– **[NW-1] Создание системы подключения игроков**

Описание: Реализовать подключение игроков к одной игровой комнате с синхронизацией данных.

– **[NW-2] Реализация обмена данными между клиентами**

Описание: Настроить передачу данных, связанных с игровым процессом (выбор тем, ответы игроков).

8.6 Задачи для подсистемы «Система управления данными»

– **[DB-1] Разработка структуры базы данных**

Описание: Создать таблицы для хранения информации об игроках, темах, результатах игры и процессе.

– **[DB-2] Реализация функционала работы с данными**

Описание: Настроить чтение и запись данных о пользователях, игровых результатах, темах.

8.7 Тестирование и отладка

– **[TST-1] Интеграционное тестирование**

Описание: Проверить взаимодействие между модулями (логика игры, интерфейс, генерация вопросов).

– **[TST-2] Отладка и исправление багов**

Описание: Провести отладку и исправить выявленные ошибки.

8.8 Сборка и выпуск

– **[RL-1] Сборка продукта**

Описание: Подготовить рабочую версию игры для выпуска.

– **[RL-2] Тестирование пользовательского опыта**

Описание: Провести тестирование с участием игроков для оценки удобства и функциональности.

– **[RL-3] Выпуск версии 1.0**

Описание: Опубликовать финальную версию продукта.

9 Разработка рекомендаций по кодированию

Для создания качественного кода на любом языке программирования, обладающего таким свойствами, как удобочитаемость (readability) и понятность (understandability), необходимо следовать хорошо определённым стандартам и руководствам. Особенно это актуально при коллективной разработке программ. Любой стандарт кодирования призван определить набор правил, которые способствуют разработке более единообразного кода и минимизации числа общераспространённых ошибок в нём, не ущемляя при этом права разработчика на творчество.

В основе рекомендаций — руководство по оформлению кода на Python PEP-8. Все правила PEP-8 должны соблюдаться в проекте. Ниже перечислены самые важные рекомендации и требования по оформлению кода.

9.1 Основные рекомендации и требования

1. Форматирование кода:

- Используйте отступы в 4 пробела. Не используйте клавишу Tab для отступов.
- Размещайте каждую инструкцию `import` в отдельной строке.
- Все импорты должны находиться в начале файла, отделены пустой строкой от остального кода.
- Избегайте использования конструкции `from module import *`.

2. Именование:

- Используйте `snake_case` для имен переменных и функций.
- Для классов используйте стиль `UpperCamelCase`.
- Константы должны быть написаны в стиле `SCREAMING_SNAKE_CASE`.
- Имена переменных и функций должны быть осмысленными и на английском языке.

- Функции и методы должны начинаться с глагола.

3. **Комментарии:**

- Используйте комментарии для пояснения сложных частей кода.
- Все комментарии должны быть на русском языке и начинаться с символа #, отделенного пробелом.

- Избегайте комментариев, противоречащих коду.

4. **Структура кода:**

- Разделяйте функции и классы двумя пустыми строками.
- Методы внутри классов отделяйте одной пустой строкой.
- Логические блоки кода внутри функций отделяйте одной пустой строкой.
- Каждый модуль проекта должен быть оформлен в отдельном файле.

5. **Работа с числами и данными:**

- Не используйте "магические числа" — присваивайте числам осмысленные константы.
- Для работы с числами используйте функции и переменные, чтобы улучшить читаемость кода.

6. **Бинарные операторы:**

- Всегда окружайте бинарные операторы (например, =, +, ==) пробелами с каждой стороны.
- Следите за логичностью использования операторов, избегайте сложных вложенных выражений.

7. **Логика и структура программ:**

- Не располагайте несколько инструкций в одной строке.
- Условные операторы (if, else) и циклы (for, while) пишите с соблюдением форматирования: каждая новая инструкция на новой строке.
- Используйте явные проверки, например, if x is None вместо if not x.

8. **Проверка длины файла:**

- Ограничивайте длину файла 250 строками, при необходимости делите код на модули.

9.2 Дополнительные рекомендации

1. Использование аннотаций типов:

- Аннотируйте типы переменных, аргументов функций и возвращаемые значения.

- Пример:

```
def calculate_sum(a: int, b: int) -> int:
    return a + b
```

2. Логирование:

- Используйте библиотеку logging вместо print для вывода сообщений отладки.

- Пример:

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Application started")
```

3. Документирование кода:

- Оформляйте документацию функций с использованием docstring.

- Пример:

```
def calculate_sum(a: int, b: int) -> int:
    """
    Calculate the sum of two integers.

    Args:
        a (int): First integer.
        b (int): Second integer.

    Returns:
        int: Sum of a and b.
    """
    return a + b
```

4. Тестирование:

– Покрывайте ключевые функции тестами, используя модуль unittest или pytest.

– Размещайте тесты в отдельной папке tests.

5. Обработка ошибок:

– Используйте блоки try-ехсепт для обработки исключений, особенно при работе с файлами или внешними источниками данных.

– Пример:

```
try:
    with open('data.txt', 'r') as file:
        data = file.read()
except FileNotFoundError:
    logging.error("File not found.")
```

9.3 Чек-лист для проверки корректности кода

– Используются отступы в 4 пробела.

– Все имена соответствуют правилам: snake_case, SCREAMING_SNAKE_CASE, UpperCamelCase.

– Отсутствуют "магические числа", все константы определены.

– Нет строк длиной более 80 символов.

– Импорты находятся в начале файла и отделены пустой строкой.

– Функции и классы разделены двумя пустыми строками.

– Все бинарные операторы окружены пробелами.

– Весь код сопровождается комментариями и docstring.

– Код протестирован и покрыт тестами.

– Логирование реализовано через библиотеку logging.

10 Разработка плана тестирования проекта

Тестирование программного продукта – это процесс проверки разрабатываемого программного средства, который включает в себя проектирование тестов, их выполнение и анализ полученных результатов, которые позволяют выявить ошибки, проверить корректность работы и оценить удобство использования.

Один из методов тестирования – это разделение входных данных на классы эквивалентности. Классы эквивалентности определяют, какие данные считаются корректными, а какие – некорректными, помогая систематизировать процесс проверки.

В таблице приведены основные входные данные, а также их отнесение к правильным и неправильным классам эквивалентности.

Т а б л и ц а 3 – Классы эквивалентности

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильно го класса эквивалентности	Неправильный класс эквивалентности
Регистрация нового пользователя	(40)	Пользователем введены корректные данные - уникальный логин и пароль. Регистрация прошла успешно. $\text{Логин}_{\text{new}} = \{l_1 l_2 \dots l_n\}$, где $l_k \in \{A \dots Z, a \dots z, 0 \dots 9, _ , -\}$, $5 \leq n \leq 20$. $\text{Пароль}_{\text{new}} = \{p_1 p_2 \dots p_n\}$, где $p_k \in \{A \dots Z, a \dots z, 0 \dots 9, \text{спецсимволы}\}$, $8 \leq n \leq 32$. Пользователь = {Логин, Пароль}	(1)	Логин уже занят: $\text{Логин}_{\text{new}} = \{l_1 l_2 \dots l_n\}$, где $l_k \in \{A \dots Z, a \dots z, 0 \dots 9, _ , -\}$, $5 \leq n \leq 20$ $\text{Логин}_{\text{new}} = \text{Логин}$
			(2)	Были введены недопустимые символы в логине: $\text{Логин} = \{l_1 l_2 \dots l_n\}$, где $l_k \notin \{A \dots Z, a \dots z, 0 \dots 9, _ , -\}$
			(3)	Недопустимая длина логина: $\text{Логин}_{\text{new}} = \{l_1 l_2 \dots l_n\}$, где $l_k \in \{A \dots Z, a \dots z, 0 \dots 9, _ , -\}$, $n < 5$
			(57)	Недопустимая длина логина: $\text{Логин}_{\text{new}} = \{l_1 l_2 \dots l_n\}$, где $l_k \in \{A \dots Z, a \dots z, 0 \dots 9, _ , -\}$, $n > 20$
			(4)	Были введены недопустимые символы в пароле: $\text{Пароль}_{\text{new}} = \{p_1 p_2 \dots p_n\}$, где $p_k \notin \{A \dots Z, a \dots z, 0 \dots 9, \text{спецсимволы}\}$
			(5)	Недопустимая длина пароля: $\text{Пароль}_{\text{new}} = \{p_1 p_2 \dots p_n\}$, где $p_k \in \{A \dots Z, a \dots z, 0 \dots 9, \text{спецсимволы}\}$, $n > 32$
			(58)	Недопустимая длина пароля: $\text{Пароль}_{\text{new}} = \{p_1 p_2 \dots p_n\}$, где $p_k \in \{A \dots Z, a \dots z, 0 \dots 9, \text{спецсимволы}\}$, $n < 8$

Продолжение Таблицы 3

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильно го класса эквивалентности	Неправильный класс эквивалентности
Вход в систему	(41)	Введены верные учетные данные - существующий логин и пароль. Пользователь вошёл в систему. Логин _{input} = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _, -}, 5 ≤ n ≤ 20. Пароль _{input} = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}, 8 ≤ n ≤ 32. Пользователь = {Логин, Пароль}	(6)	Такого логина не существует: Логин _{input} = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _, -}, 5 ≤ n ≤ 20 Логин _{input} ≠ Логин
			(7)	Неверный пароль: Пароль _{input} = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}, 8 ≤ n ≤ 32 Пароль _{input} ≠ Пароль
			(8)	Пустые поля: Логин _{input} = ∅ Пароль _{input} = ∅
			(9)	Были введены недопустимые символы в логине: Логин _{input} = {l ₁ l ₂ ...l _n }, где l _k ∉ {A...Z, a...z, 0...9, _, -}
			(10)	Недопустимая длина логина: Логин _{input} = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _, -}, n < 5
			(54)	Недопустимая длина логина: Логин _{input} = {l ₁ l ₂ ...l _n }, где l _k ∈ {A...Z, a...z, 0...9, _, -}, n > 20
			(11)	Были введены недопустимые символы в пароле: Пароль _{input} = {p ₁ p ₂ ...p _n }, где p _k ∉ {A...Z, a...z, 0...9, спецсимволы}
			(12)	Недопустимая длина пароля: Пароль _{input} = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}, n < 8
			(55)	Недопустимая длина пароля: Пароль _{input} = {p ₁ p ₂ ...p _n }, где p _k ∈ {A...Z, a...z, 0...9, спецсимволы}, n > 32

Продолжение Таблицы 3

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильно го класса эквивалентности	Неправильный класс эквивалентности
Создание комнаты	(42)	Комната успешно создана	(13)	Ошибка в создании комнаты
Подключение к комнате	(43)	Игрок успешно подключился к комнате	(14)	Игрок без авторизации
			(15)	Комната переполнена
			(16)	Комната не существует
Запуск игры	(44)	Игра успешно запущена владельцем комнаты	(17)	Игрок, не являющимся владельцем, пытается запустить комнату
			(18)	Недостаточно игроков
Начисление очков	(46)	Верный подсчет (правильный ответ = +1 очко)	(20)	Некорректное начисление (ошибочное уменьшение, пропуск начисления)
Формирование запроса для генерации вопросов и ответов	(47)	Отправка корректного запроса, в котором указан режим игры и тема. Запрос = $\{v_1v_2...v_n, mode\}$, где $v_1 \in \{A...Я, A...Z\}$, $v_k \in \{A...Я, а...я, A...Z, а...z, 0...9, пробел, спецсимволы\}$, $1 \leq n \leq 200$, $mode = \{\text{«знание»}, \text{«абсурд»}\}$	(21)	Пустой запрос: Запрос = $\emptyset$
			(22)	Запрос сформулирован некорректно: Запрос = $\{v_1v_2...v_n, mode\}$, где $v_1 \notin \{A...Я, A...Z\}$ или $v_k \notin \{A...Я, а...я, A...Z, а...z, 0...9, пробел, спецсимволы\}$
			(23)	Недопустимая длина: Запрос = $\{v_1v_2...v_n, mode\}$, где $v_1 \in \{A...Я, A...Z\}$, $v_k \in \{A...Я, а...я, A...Z, а...z, 0...9, пробел, спецсимволы\}$, $mode = \{\text{«знание»}, \text{«абсурд»}\}$, $n > 200$
			(56)	Недопустимая длина: Запрос = $\{v_1v_2...v_n, mode\}$, где $v_1 \in \{A...Я, A...Z\}$, $v_k \in \{A...Я, а...я, A...Z, а...z, 0...9, пробел, спецсимволы\}$, $mode = \{\text{«знание»}, \text{«абсурд»}\}$, $n < 1$
			(53)	Не задан режим: Запрос = $\{v_1v_2...v_n, mode\}$, где $v_1 \in \{A...Я, A...Z\}$, $v_k \in \{A...Я, а...я, A...Z, а...z, 0...9, пробел, спецсимволы\}$, $mode = \emptyset$

Продолжение Таблицы 3

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильно го класса эквивалентности	Неправильный класс эквивалентности
Генерация вопросов и ответов (режим знаний)	(48)	Формирование списка вопросов с ответами на них. Каждый вопрос соответствует заданной теме, которую пользователь выбрал из предложенного ему списка при создании комнаты, корректно сформулирован, и имеет единственно верный правильный ответ. Каждый список вариантов ответов на вопрос состоит из 4 вариантов, один из которых является правильным ответом. Все варианты ответов корректно сформулированы. Вопрос = $\{v_1 v_2 \dots v_n\}$, где $v_1 \in \{A \dots Я\}$, $v_k \in \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$, $10 \leq n \leq 150$. Ответ = $\{q_1 q_2 \dots q_n\}$, где $q_1 \in \{A \dots Я, 0 \dots 9\}$, $q_k \in \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел}\}$, $1 \leq n \leq 20$. Список вопросов = $\{(\text{Вопрос}_1, \text{Варианты ответов}_1), (\text{Вопрос}_2, \text{Варианты ответов}_2), \dots, (\text{Вопрос}_n, \text{Варианты ответов}_n)\}$, где $n \in \mathbb{N}$ Варианты ответов = $\{(\text{Ответ}_1, \text{flag}_1) \dots (\text{Ответ}_4, \text{flag}_4)\}$, где $\text{flag}_k = \{0, 1\}$	(24)	Вопросы не соответствуют заданной теме
			(25)	Вопросы сформулированы некорректно, содержат ошибки: Вопрос = $\{v_1 v_2 \dots v_n\}$, где $v_1 \notin \{A \dots Я\}$ или $v_k \notin \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$
			(26)	Вопросы повторяются: Список вопросов = $\{(\text{Вопрос}_1, \text{Варианты ответов}_1), (\text{Вопрос}_2, \text{Варианты ответов}_2), \dots, (\text{Вопрос}_n, \text{Варианты ответов}_n)\}$, где $(\text{Вопрос}_k, \text{Варианты ответов}_k) = (\text{Вопрос}_i, \text{Варианты ответов}_i)$
			(27)	Варианты ответов сформулированы некорректно: Ответ = $\{q_1 q_2 \dots q_n\}$, где $q_1 \notin \{A \dots Я, 0 \dots 9\}$ или $q_k \notin \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$
			(28)	Среди вариантов ответов на вопрос нет правильного Варианты ответов = $\{(\text{Ответ}_1, 0), (\text{Ответ}_2, 0), (\text{Ответ}_3, 0), (\text{Ответ}_4, 0)\}$
			(29)	Повторение вариантов ответов в одном вопросе: Варианты ответов = $\{(\text{Ответ}_1, \text{flag}_1), \dots, (\text{Ответ}_4, \text{flag}_4)\}$, где $(\text{Ответ}_k, \text{flag}_k) = (\text{Ответ}_i, \text{flag}_i)$

Продолжение Таблицы 3

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильно го класса эквивалентности	Неправильный класс эквивалентности
Генерация вопросов (абсурдный режим)	(49)	Формирование списка вопросов с ответами на них. Каждый вопрос соответствует заданной теме, которую пользователь вводит сам при создании комнаты, корректно сформулирован, и имеет единственно верный правильный ответ. Каждый список вариантов ответов на вопрос состоит из 4 вариантов, один из которых является правильным ответом. Все варианты ответов корректно сформулированы. Вопрос = $\{v_1 v_2 \dots v_n\}$, где $v_1 \in \{A \dots Я\}$, $v_k \in \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$, $10 \leq n \leq 150$. Ответ = $\{q_1 q_2 \dots q_n\}$, где $q_1 \in \{A \dots Я, 0 \dots 9\}$, $q_k \in \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$, $1 \leq n \leq 20$. Список вопросов = $\{(\text{Вопрос}_1, \text{Варианты ответов}_1), (\text{Вопрос}_2, \text{Варианты ответов}_2), \dots, (\text{Вопрос}_n, \text{Варианты ответов}_n)\}$, где $n \in \mathbb{N}$ Варианты ответов = $\{(\text{Ответ}_1, \text{flag}_1), \dots, (\text{Ответ}_4, \text{flag}_4)\}$, где $\text{flag} = \{0, 1\}$	(30)	Вопросы не соответствуют заданной теме
			(31)	Вопросы сформулированы некорректно, содержат ошибки: Вопрос = $\{v_1 v_2 \dots v_n\}$, где $v_1 \notin \{A \dots Я\}$ или $v_k \notin \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$
			(32)	Вопросы повторяются: Список вопросов = $\{(\text{Вопрос}_1, \text{Варианты ответов}_1), (\text{Вопрос}_2, \text{Варианты ответов}_2), \dots, (\text{Вопрос}_n, \text{Варианты ответов}_n)\}$, где $(\text{Вопрос}_k, \text{Варианты ответов}_k) = (\text{Вопрос}_i, \text{Варианты ответов}_i)$
			(33)	Варианты ответов сформулированы некорректно: Ответ = $\{q_1 q_2 \dots q_n\}$, где $q_1 \notin \{A \dots Я, 0 \dots 9\}$ или $q_k \notin \{A \dots Я, a \dots я, A \dots Z, a \dots z, 0 \dots 9, \text{пробел, спецсимволы}\}$
			(34)	Среди вариантов ответов на вопрос нет правильного: Варианты ответов = $\{(\text{Ответ}_1, 0), (\text{Ответ}_2, 0), (\text{Ответ}_3, 0), (\text{Ответ}_4, 0)\}$
Ответ на вопрос	(45)	Выбран один из четырёх вариантов ответа	(52)	Повторение вариантов ответов в одном вопросе: Варианты ответов = $\{(\text{Ответ}_1, \text{flag}_1), \dots, (\text{Ответ}_4, \text{flag}_4)\}$, где $(\text{Ответ}_k, \text{flag}_k) = (\text{Ответ}_i, \text{flag}_i)$
			(19)	Не выбран ни один вариант

Окончание Таблицы 3

Входные данные	Номер правильного класса эквивалентности	Правильный класс эквивалентности	Номер неправильно го класса эквивалентности	Неправильный класс эквивалентности
Завершение игры	(50)	Финальные очки подсчитаны и определен победитель Таблица результатов = $\{(L_1, score_1), \dots, (L_n, score_n)\}$, где n – кол-во игроков, $0 \leq score \leq$ кол-во вопросов	(35)	Ошибки в подсчёте очков: Таблица результатов = $\{(L_1, score_1), \dots, (L_n, score_n)\}$, где n – кол-во игроков, $score < 0$ или $score >$ кол-во вопросов
			(36)	Игра не завершается корректно
Отправка сообщения в чат	(51)	Сообщение отправлено корректно. $M = \{m_1 m_2 \dots m_n\}$, где $m_k \in \{A \dots Z, a \dots z, A \dots Я, а \dots я, 0 \dots 9, \text{пробел, знаки препинания, эмодзи, спецсимволы}\}$, $1 \leq n \leq 500$	(37)	Сообщение пустое: $M = \emptyset$
			(38)	Сообщение превышает допустимую длину: $M = \{m_1 m_2 \dots m_n\}$, где $m_k \in \{A \dots Z, a \dots z, A \dots Я, а \dots я, 0 \dots 9, \text{пробел, знаки препинания, эмодзи, спецсимволы}\}$, $n > 500$
			(39)	Сообщение содержит недопустимые символы: $M = \{m_1 m_2 \dots m_n\}$, где $m_k \notin \{A \dots Z, a \dots z, A \dots Я, а \dots я, 0 \dots 9, \text{пробел, знаки препинания, эмодзи, спецсимволы}\}$

11 Тестирование проекта

В таблицах 4–22 представлены тесты для покрытия классов эквивалентностей.

Таблица 4 – Тест 1

№	Входные данные	
1	Новый пользователь с логином «IvanKh», паролем «1234abcd», почтой «ivan_mail@mail.ru»	
	Выходные данные	Покрытые классы эквивалентности
	Успешная регистрация	40

Таблица 5 – Тест 2

№	Входные данные	
2	Новый пользователь с логином «Ivan@Kh», паролем «1234», почтой «ivaru»	
	Выходные данные	Покрытые классы эквивалентности
	Регистрация не удалась	1,2,3,4,5, 57, 58

Таблица 6 – Тест 3

№	Входные данные	
3	Пользователь с логином «IvanKh» и паролем «1234abcd»	
	Выходные данные	Покрытые классы эквивалентности
	Успешный вход в систему	41, 6, 7

Таблица 7 – Тест 4

№	Входные данные	
4	Пользователь с логином «Ivan» и паролем «1234»	
	Выходные данные	Покрытые классы эквивалентности
	Ошибка авторизации	9, 10, 11, 12, 54, 55

Таблица 8 – Тест 5

№	Входные данные	
5	Пользователь с логином «>» и паролем «>»	
	Выходные данные	Покрытые классы эквивалентности
	Ошибка авторизации	8

Таблица 9 – Тест 6

№	Входные данные	
6	Создание комнаты	
	Выходные данные	Покрытые классы эквивалентности
	-	42,137

Таблица 10 – Тест 7

№	Входные данные	
7	Авторизованный пользователь	
	Выходные данные	Покрытые классы эквивалентности
	Успешное подключение к комнате	43

Таблица 11 – Тест 8

№	Входные данные	
8	Авторизованный пользователь	
	Выходные данные	Покрытые классы эквивалентности
	Подключение к комнате не удалось	14, 15, 16

Таблица 12 – Тест 9

№	Входные данные	
9	Владелец комнаты, в комнате нужное число игроков	
	Выходные данные	Покрытые классы эквивалентности
	Успешный запуск игры	44

Таблица 13 – Тест 10

№	Входные данные	
10	Владелец комнаты, в комнате недостаточно игроков	
	Выходные данные	Покрытые классы эквивалентности
	-	17

Таблица 14 – Тест 11

№	Входные данные	
11	Игрок	
	Выходные данные	Покрытые классы эквивалентности
	-	18

Таблица 15 – Тест 12

№	Входные данные	
12	Ответ на вопрос	
	Выходные данные	Покрытые классы эквивалентности
	-	45, 46, 19, 20

Таблица 16 – Тест 13

№	Входные данные	
13	Запрос = {Капибары, абсурд}	
	Выходные данные	Покрытые классы эквивалентности
	Массив вопросов и ответов	47, 48, 49, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 52

Таблица 17 – Тест 14

№	Входные данные	
14	Запрос = {лгг\$2#gk, знание}	
	Выходные данные	Покрытые классы эквивалентности
	Сообщение об ошибке	22

Таблица 18 – Тест 15

№	Входные данные	
15	Запрос = { , }	
	Выходные данные	Покрытые классы эквивалентности
	Сообщение об ошибке	21, 23, 53, 56

Таблица 19 – Тест 16

№	Входные данные	
16	Таблица с результатами игры	
	Выходные данные	Покрытые классы эквивалентности
	-	50, 35, 36

Таблица 20 – Тест 17

№	Входные данные	
17	Сообщение в чате: «Привет!»	
	Выходные данные	Покрытые классы эквивалентности
	Сообщение отправлено успешно	51

Таблица 21 – Тест 18

№	Входные данные	
18	Сообщение в чате: «»	
	Выходные данные	Покрытые классы эквивалентности
	Сообщение не отправлено	37

Т а б л и ц а 22 – Тест 19

№	Входные данные	
1 9	Сообщение в чате: «...∈...®...ℝ...€ ...рущопттуклдафпыпклдфьпдлфдлмьфудлклп)(*?:%;ацщашщс.....», длина сообщения больше 500 символов	
	Выходные данные	Покрытые классы эквивалентности
	Сообщение отправлено успешно	38, 39

Заключение

По окончании всех работ были вычислены метрики, описанные в 7 главе:

1. Количество дней на обработку задачи (PRR) составило 21 день;
2. Эффективность обнаружения дефектов (FS) составила 20%;
3. Плотность неполадок (IPF) составила 0,01/LOC.

В рамках курсовой работы было разработано программное средство «Интеллектуально-развлекательная игра “Сила AI”» с использованием подходов коллективной промышленной разработки, для чего были решены следующие поставленные задачи:

- Разработан план проекта;
- Разработан регламент проведения инспекции;
- Разработана модель состояний задач;
- Разработана презентация проекта;
- Разработаны требования к проекту;
- Разработана архитектура проекта;
- Разработать измерения проекта;
- Разработан перечень задач проекта;
- Разработаны рекомендации по кодированию;
- Разработан план тестирования проекта;
- Проект протестирован.

Таким образом, цель данного курсовой работы была достигнута.

Список литературы

1. Гриняк В.М. Лекции по дисциплине «Технологии коллективной промышленной разработки информационных систем». Электронный вариант.